Machine Learning Experiment Number 3

NAME - Vidya Rajan Dharne .
ROLL NO - CSE12

Aim : To implement a Multiple Linear Regression model from scratch using NumPy matrix operations to predict the selling price of used cars using multiple features from the CarDekho Used Car Dataset.

Learning Outcomes • Understand the working principle of Multiple Linear Regression.

• Perform data preprocessing and feature engineering.

• Convert categorical data into numerical features.

• Standardize numerical features.

• Implement the Ordinary Least Squares method using matrix operations.

• Train and test a regression model without using a ready-made regression function.

• Evaluate and interpret regression performance using RSS and R^2.

• Understand the effect of individual features on the predicted price.

Dataset

Use the **CarDekho Used Car Dataset** (`car data.csv`). CarDekho Used Car Dataset

Explain meaning of each attribute in this dataset in below comment section :

Here, **Selling_Price** is the target variable.

Software/Tools Required

- Python 3.x
- Google Colab / Jupyter Notebook
- NumPy
- Matplotlib

Task 1: Set Up the Python Environment**

- Create a new Google Colab notebook.
- Import **NumPy** for numerical and matrix operations.
- Import **Matplotlib** for data visualization.
- Do not use ready-made machine-learning regression functions such as `LinearRegression()` from Scikit-learn.
- The regression model must be implemented using matrix operations.

In [ ]:
```
#Import NumPy for numerical and matrix operations.
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Task 2: Load and Inspect the Dataset

- Download the `car data.csv` file.
- Upload the dataset to Google Colab.
- Load the dataset into Python.
- Display the first few records.
- Determine the number of rows and columns.
- Identify the numerical and categorical attributes.
- Separate `Selling_Price` as the target variable.

In [ ]:
```python
from google.colab import files
import pandas as pd
import numpy as np
import io

uploaded = files.upload()  # select car data.csv when prompted

# Get the filename that was actually uploaded
filename = list(uploaded.keys())[0]
print("Uploaded file:", filename)

df = pd.read_csv(io.BytesIO(uploaded[filename]))

print(df.head())
print("\nNumber of rows:", df.shape[0])
print("Number of columns:", df.shape[1])

numerical_cols = df.select_dtypes(include=[np.number]).columns.tolist()
categorical_cols = df.select_dtypes(include=['object']).columns.tolist()
print("\nNumerical columns:", numerical_cols)
print("Categorical columns:", categorical_cols)

target = "Selling_Price"
print("\nTarget variable:", target)
```

Choose Files No file selected Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving car data.csv.csv to car data.csv (2).csv
Uploaded file: car data.csv (2).csv
   Unnamed: 0       car_name    brand     model  vehicle_age  km_driven  \
0           0    Maruti Alto   Maruti      Alto            9     120000
1           1  Hyundai Grand  Hyundai     Grand            5      20000
2           2    Hyundai i20  Hyundai       i20           11      60000
3           3    Maruti Alto   Maruti      Alto            9      37000
4           4  Ford Ecosport     Ford  Ecosport            6      30000

  seller_type fuel_type transmission_type  mileage  engine  max_power  seats  \
0  Individual    Petrol            Manual    19.70     796      46.30      5
1  Individual    Petrol            Manual    18.90    1197      82.00      5
2  Individual    Petrol            Manual    17.00    1197      80.00      5
3  Individual    Petrol            Manual    20.92     998      67.10      5
4      Dealer    Diesel            Manual    22.77    1498      98.59      5

   selling_price
0         120000
1         550000
2         215000
3         226000
4         570000

Number of rows: 15411
Number of columns: 14

Numerical columns: ['Unnamed: 0', 'vehicle_age', 'km_driven', 'mileage', 'engine', 'max_power', 'seats',
'selling_price']
Categorical columns: ['car_name', 'brand', 'model', 'seller_type', 'fuel_type', 'transmission_type']

Target variable: Selling_Price
```

Task 3: Check and Handle Missing Data

- Check every attribute for missing or null values.
- Count the number of missing values in each column.
- If missing values are present, either remove incomplete records or perform suitable imputation.
- Verify that the final dataset does not contain missing values.
- Record the preprocessing method used.

In [ ]:
```python
print("Missing values per column:\n", df.isnull().sum())

# CarDekho dataset is usually clean, but drop any incomplete rows just in case
df = df.dropna().reset_index(drop=True)

print("\nAfter cleaning, missing values:\n", df.isnull().sum())
print("Final shape:", df.shape)
```

```
Missing values per column:
 Unnamed: 0           0
car_name              0
brand                 0
model                 0
vehicle_age           0
km_driven             0
seller_type           0
fuel_type             0
transmission_type     0
mileage               0
engine                0
max_power             0
seats                 0
selling_price         0
dtype: int64

After cleaning, missing values:
 Unnamed: 0           0
car_name              0
brand                 0
model                 0
vehicle_age           0
km_driven             0
seller_type           0
fuel_type             0
transmission_type     0
mileage               0
engine                0
max_power             0
seats                 0
selling_price         0
dtype: int64
Final shape: (15411, 14)
```

Task 4: Select and Prepare Features

- Examine all available attributes.
- Decide how `Car_Name` should be handled.
- Select meaningful predictor variables for the regression model.
- Create the predictor matrix X and target vector y.
- Record the final list of features selected for the model.

In [ ]:
```python
# Clean up column names (removes accidental leading/trailing spaces)
df.columns = df.columns.str.strip()

print("Columns:", df.columns.tolist())

# Drop Car_Name only if it exists, to avoid errors
if "Car_Name" in df.columns:
    df = df.drop(columns=["Car_Name"])

print(df.columns.tolist())
```

```
Columns: ['Unnamed: 0', 'car_name', 'brand', 'model', 'vehicle_age', 'km_driven', 'seller_type', 'fuel_type',
'transmission_type', 'mileage', 'engine', 'max_power', 'seats', 'selling_price']
['Unnamed: 0', 'car_name', 'brand', 'model', 'vehicle_age', 'km_driven', 'seller_type', 'fuel_type',
'transmission_type', 'mileage', 'engine', 'max_power', 'seats', 'selling_price']
```

Task 5: Encode Categorical Variables

Identify the categorical variables:

- `Fuel_Type`
- `Seller_Type`
- `Transmission`

Perform the following:

- Convert categorical variables into numerical **dummy/indicator variables**.
- Verify that all predictors are numerical after encoding.
- Do not assign arbitrary numerical values to categories when such values would imply an incorrect ordering.
- Display the transformed dataset.

In [ ]:

```
# Clean up column names (strip spaces)
df.columns = df.columns.str.strip()
print("Available columns:", df.columns.tolist())

# Build a case-insensitive lookup to find the real column names
col_lookup = {c.lower().replace(" ", "").replace("_", ""): c for c in df.columns}

target_names = ["Fuel_Type", "Seller_Type", "Transmission"]
categorical_vars = []

for name in target_names:
    key = name.lower().replace("_", "")
    if key in col_lookup:
        categorical_vars.append(col_lookup[key])
    else:
        print(f"Warning: could not find a column matching '{name}'")

print("\nMatched categorical columns:", categorical_vars)

# Convert categorical variables into numerical dummy/indicator variables
df_encoded = pd.get_dummies(df, columns=categorical_vars, drop_first=True)

# Convert boolean dummy columns to int (0/1)
bool_cols = df_encoded.select_dtypes(include='bool').columns
df_encoded[bool_cols] = df_encoded[bool_cols].astype(int)

print("\nData types after encoding:\n", df_encoded.dtypes)
print("\nTransformed dataset:")
print(df_encoded.head())
```

```
Available columns: ['Unnamed: 0', 'car_name', 'brand', 'model', 'vehicle_age', 'km_driven', 'seller_type',
'fuel_type', 'transmission_type', 'mileage', 'engine', 'max_power', 'seats', 'selling_price']
Warning: could not find a column matching 'Transmission'

Matched categorical columns: ['fuel_type', 'seller_type']

Data types after encoding:
 Unnamed: 0                      int64
car_name                        object
brand                           object
model                           object
vehicle_age                      int64
km_driven                        int64
transmission_type               object
mileage                        float64
engine                           int64
max_power                      float64
seats                            int64
selling_price                    int64
fuel_type_Diesel                 int64
fuel_type_Electric               int64
fuel_type_LPG                    int64
fuel_type_Petrol                 int64
seller_type_Individual           int64
seller_type_Trustmark Dealer     int64
dtype: object

Transformed dataset:
   Unnamed: 0       car_name    brand     model  vehicle_age  km_driven  \
0           0    Maruti Alto   Maruti      Alto            9     120000
1           1  Hyundai Grand  Hyundai     Grand            5      20000
2           2    Hyundai i20  Hyundai       i20           11      60000
3           3    Maruti Alto   Maruti      Alto            9      37000
4           4  Ford Ecosport     Ford  Ecosport            6      30000

  transmission_type  mileage  engine  max_power  seats  selling_price  \
0            Manual    19.70     796      46.30      5         120000
1            Manual    18.90    1197      82.00      5         550000
2            Manual    17.00    1197      80.00      5         215000
3            Manual    20.92     998      67.10      5         226000
4            Manual    22.77    1498      98.59      5         570000

   fuel_type_Diesel  fuel_type_Electric  fuel_type_LPG  fuel_type_Petrol  \
0                 0                   0              0                 1
1                 0                   0              0                 1
2                 0                   0              0                 1
3                 0                   0              0                 1
4                 1                   0              0                 0

   seller_type_Individual  seller_type_Trustmark Dealer
0                       1                             0
1                       1                             0
2                       1                             0
3                       1                             0
4                       0                             0
```

Task 6: Perform Feature Engineering

Create a new feature called `Car_Age`.

Calculate it using:

[Car_Age = Current Year - Year]

Perform the following:

- Create the `Car_Age` column.
- Display several values of the newly created feature.
- Verify that the calculated ages are reasonable.
- Decide whether the original `Year` feature should be retained.
- Provide a reason for your decision.

In [ ]:
```
# Task 6: Perform Feature Engineering

# Create Car_Age from the existing vehicle_age feature
```

```
df_encoded['Car_Age'] = df_encoded['vehicle_age']

# Display several values of the newly created feature
print("Car_Age values:")
display(df_encoded[['vehicle_age', 'Car_Age']].head(10))

# Verify that the calculated ages are reasonable
print("\nCar_Age Statistics:")
print(df_encoded['Car_Age'].describe())

# Check minimum and maximum age
print("\nMinimum Car_Age:", df_encoded['Car_Age'].min())
print("Maximum Car_Age:", df_encoded['Car_Age'].max())

# Decision regarding the original vehicle_age feature
print("\nDecision: Original 'vehicle_age' feature should be removed.")
print("Reason: Car_Age contains the same information as vehicle_age,")
print("so retaining both would create a duplicate feature.")

# Remove the original vehicle_age feature
df_encoded = df_encoded.drop(columns=['vehicle_age'])

# Display updated dataset
print("\nDataset after feature engineering:")
display(df_encoded.head())
```

```
Car_Age values:
```

	vehicle_age	Car_Age
0	9	9
1	5	5
2	11	11
3	9	9
4	6	6
5	8	8
6	8	8
7	3	3
8	2	2
9	4	4

```
Car_Age Statistics:
count    15411.000000
mean         6.036338
std          3.013291
min          0.000000
25%          4.000000
50%          6.000000
75%          8.000000
max         29.000000
Name: Car_Age, dtype: float64

Minimum Car_Age: 0
Maximum Car_Age: 29

Decision: Original 'vehicle_age' feature should be removed.
Reason: Car_Age contains the same information as vehicle_age,
so retaining both would create a duplicate feature.

Dataset after feature engineering:
```

	Unnamed: 0	car_name	brand	model	km_driven	transmission_type	mileage	engine	max_power	seats	selling_price	fuel_type_Diesel	fuel_type_Electric	fuel_type_LPG	fuel_type_Petrol	seller_type_Individual	seller_type_Trustmark Dealer	Car_Age
0	0	Maruti Alto	Maruti	Alto	120000	Manual	19.70	796	46.30	5	120000	0	0	0	1	1	0	9
1	1	Hyundai Grand	Hyundai	Grand	20000	Manual	18.90	1197	82.00	5	550000	0	0	0	1	1	0	5
2	2	Hyundai i20	Hyundai	i20	60000	Manual	17.00	1197	80.00	5	215000	0	0	0	1	1	0	11
3	3	Maruti Alto	Maruti	Alto	37000	Manual	20.92	998	67.10	5	226000	0	0	0	1	1	0	9
4	4	Ford Ecosport	Ford	Ecosport	30000	Manual	22.77	1498	98.59	5	570000	1	0	0	0	0	0	6

Task 7: Perform Exploratory Data Analysis

Perform basic exploratory data analysis using Matplotlib.

- Create a scatter plot of `Present_Price` versus `Selling_Price`.
- Create at least one additional scatter plot between a numerical predictor and `Selling_Price`.
- Label the axes appropriately.
- Add suitable titles to the plots.
- Observe the direction and approximate strength of the relationships.
- Record your observations.

In [ ]:
```
# Debug: show what's actually available
print("Available columns:", df_encoded.columns.tolist())

# Auto-detect the relevant columns
def find_col(possible_keywords, columns):
    for c in columns:
        if any(k in c.lower().replace(" ", "").replace("_", "") for k in possible_keywords):
            return c
    return None

present_price_col = find_col(["presentprice"], df_encoded.columns)
selling_price_col = find_col(["sellingprice"], df_encoded.columns)
car_age_col = find_col(["carage"], df_encoded.columns)

print("Present_Price column found:", present_price_col)
print("Selling_Price column found:", selling_price_col)
print("Car_Age column found:", car_age_col)

if present_price_col is None or selling_price_col is None:
    raise ValueError("Could not find Present_Price / Selling_Price columns. Check the printed column list
above.")

# Plot 1: Present Price vs Selling Price
plt.figure(figsize=(6,4))
plt.scatter(df_encoded[present_price_col], df_encoded[selling_price_col], alpha=0.6, color="teal")
plt.xlabel("Present Price (in lakhs)")
plt.ylabel("Selling Price (in lakhs)")
plt.title("Present Price vs Selling Price")
plt.show()

# Plot 2: Car Age vs Selling Price (only if Car_Age exists)
if car_age_col is not None:
    plt.figure(figsize=(6,4))
    plt.scatter(df_encoded[car_age_col], df_encoded[selling_price_col], alpha=0.6, color="orange")
    plt.xlabel("Car Age (years)")
    plt.ylabel("Selling Price (in lakhs)")
    plt.title("Car Age vs Selling Price")
    plt.show()
else:
    print("Car_Age column not found — make sure Task 6 (creating Car_Age) ran successfully before this
cell.")
```

```
Available columns: ['Unnamed: 0', 'car_name', 'brand', 'model', 'km_driven', 'transmission_type', 'mileage',
'engine', 'max_power', 'seats', 'selling_price', 'fuel_type_Diesel', 'fuel_type_Electric', 'fuel_type_LPG',
'fuel_type_Petrol', 'seller_type_Individual', 'seller_type_Trustmark Dealer', 'Car_Age']
Present_Price column found: None
Selling_Price column found: selling_price
Car_Age column found: Car_Age
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_549/734958755.py in <cell line: 0>()
     18
     19 if present_price_col is None or selling_price_col is None:
---> 20     raise ValueError("Could not find Present_Price / Selling_Price columns. Check the printed column
list above.")
     21
     22 # Plot 1: Present Price vs Selling Price

ValueError: Could not find Present_Price / Selling_Price columns. Check the printed column list above.
```

Task 8: Standardize Numerical Features

Identify the numerical predictors that need to be standardized.

Use the following standardization equation:

[z = (x-μ)/σ]

where:

- x = original feature value
- μ = mean of the feature
- σ = standard deviation of the feature

Perform the following:

- Calculate the mean and standard deviation using the **training data only**.
- Standardize the training features.
- Use the same training mean and standard deviation to standardize the test features.
- Do not standardize the target variable `Selling_Price`.

In [ ]:

Task 9: Split the Dataset

Divide the dataset into training and testing subsets.

- Use an **80:20 train-test ratio**.
- Randomly select the observations.
- Set a fixed random seed to make the experiment reproducible.
- Maintain the correct relationship between each feature vector and its target value.
- Display the number of observations in the training and testing sets.

In [ ]:

Task 10: Construct the Feature Matrix

Construct the matrices required for regression.

Create:

- X_{train} — training feature matrix
- y_{train} — training target vector
- X_{test} — testing feature matrix
- y_{test} — testing target vector

Verify:

- Number of rows in each matrix.
- Number of predictor columns.
- Shape of the training and testing matrices.

In [ ]:

Task 11: Add the Bias/Intercept Term

Add a leading column containing 1.0 to both the training and testing feature matrices.

The resulting matrices should contain:

[X = \begin{bmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1p}\\ 1 & x_{21} & x_{22} & \cdots & x_{2p}\\ \vdots & \vdots & \vdots & \ddots & \vdots\\ 1 & x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix}]

Perform the following:

- Add the bias column.

- Verify the new matrix dimensions.
- Understand that the first column represents the intercept β_0.

In [ ]:

Task 12: Implement the OLS Function

Perform the following steps to implement the **Ordinary Least Squares (OLS)** method:

- Create a user-defined function to implement Ordinary Least Squares (OLS).
- Calculate the matrix X^TX.
- Calculate the matrix $X^T y$.
- Estimate the regression coefficient vector using the normal equation:
$\hat{\beta} = (X^TX)^{-1}X^Ty$.

- Return the calculated coefficient vector.
- Use **NumPy matrix operations** to perform the calculations.
- Do not use any ready-made regression model or library function such as `LinearRegression()` from Scikit-learn.\

In [ ]:

Task 13: Check Matrix Singularity

- Calculate the determinant of $X^T X$.
- Check whether the determinant is zero or sufficiently close to zero.
- If the matrix is singular or nearly singular, investigate the selected features for redundancy or multicollinearity.
- Modify the feature set if required.
- Record your observations.

In [ ]:

Task 14: Train the Regression Model

- Apply the OLS function to the training dataset.
- Obtain the regression coefficient vector $\hat{\beta}$.
- Display the intercept coefficient β_0.
- Display the coefficient corresponding to each feature.
- Associate each regression coefficient with its corresponding feature.

In [ ]:

Task 15: Interpret the Regression Coefficients

- Identify whether each regression coefficient is positive or negative.
- Explain what the sign of each coefficient indicates.
- Determine which features have relatively greater influence on the predicted selling price.
- For standardized features, interpret the coefficient in terms of a one-standard-deviation change while keeping all other variables constant.

In [ ]:

Task 16: Predict Selling Prices

- Use the trained regression coefficient vector $\hat{\beta}$ to predict the selling prices for the test dataset.
- Calculate the predicted values using the equation: $\hat{y}_{test} = X_{test}\hat{\beta}$.
- Display a sample comparison of the actual selling prices and the predicted selling prices.

In [ ]:

Task 17: Calculate Prediction Errors and RSS

- Calculate the residual for each test observation using: $e_i = y_i - \hat{y}_i$.
- Calculate the **Residual Sum of Squares (RSS): $RSS = \Sigma(y_i - \hat{y}_i)^2$.
- Display the calculated RSS value.
- Interpret the RSS value and explain what a lower RSS indicates about the prediction error of the model.

In [ ]:

Task 18: Calculate and Interpret R^2

• Calculate the Total Sum of Squares (TSS).

• Calculate $R^2 = 1 - RSS/TSS$.

• Display the R^2 value.

• Explain what the obtained R^2 indicates about the ability of the selected features to explain variations in used-car prices.

In [ ]:

Task 19: Visualize and Analyze Model Performance

• Create an Actual Selling Price vs Predicted Selling Price plot.

• Add a reference line representing perfect prediction.

• Plot or examine the residuals.

• Identify observations with relatively large prediction errors.

• Comment on whether the errors appear randomly distributed.

• Discuss possible reasons for prediction errors.

In [ ]:

Task 20: Conclude

• Comment on the most influential features.

• Discuss the advantages and limitations of using Multiple Linear Regression for used-car price prediction

Conclusion

Conclude the experiment by stating whether the Multiple Linear Regression model implemented using the OLS normal equation was able to predict used-car selling prices effectively. The conclusion should include the obtained RSS and R^2 values, observations regarding important features, the effect of preprocessing and feature engineering, and the limitations of using a linear regression model for real-world used-car price prediction.